

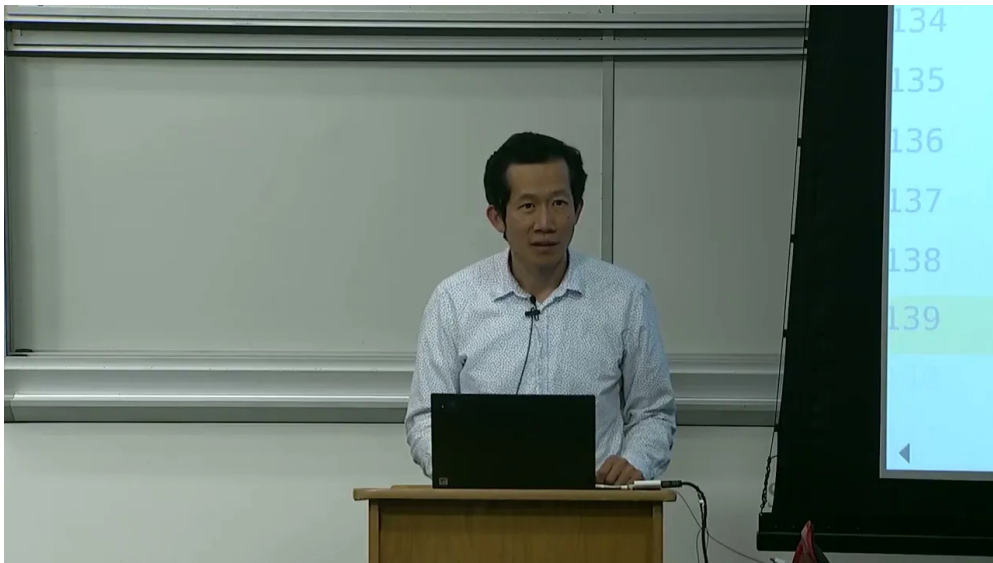
课程笔记

CS336 Lecture 10: 语言模型推理

Stanford CS336: Language Modeling from Scratch (Spring 2025)

讲师: Percy Liang, Tatsu Hashimoto 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 82 分钟

视频链接: <https://www.youtube.com/watch?v=fcgPYo30tV0>

Contents

1	为什么要严肃对待推理	2
1.1	推理性能的四个关键指标	2
1.2	统一符号定义	3
2	算术强度：推理优化的统一框架	4
2.1	矩阵乘法的 FLOPs 与字节	4
2.2	算术强度的定义	5
2.3	H100 的 Roofline 阈值	6
2.4	$B = 1$ ：典型的带宽受限场景	6
3	KV Cache：避免重复计算	7
3.1	朴素实现的冗余	7
3.2	KV Cache 的核心思想	8
3.3	推理的两个阶段：Prefill 与 Decode	9
4	注意力的算术强度推导	11
4.1	Decode 阶段的注意力： $\sim ST/(S+T)$	11
4.2	当 $T \gg S$ ：算术强度趋于 1	11
5	Llama2 手算：把公式落到具体数字	12
5.1	批量：把吞吐拉上去	13
5.2	$B = 256$ ：KV cache 把显存撑爆	15
5.3	复制与张量并行：跨卡扩展	15
6	改造架构：降低推理的固有成本	16
6.1	标准 MHA：KV cache 的代价	17
6.2	MQA：所有头共享一组 KV	18
6.3	GQA：折中方案	18
6.4	MLA：DeepSeek 的低秩压缩	19
6.5	局部注意力与跨层共享	20
6.6	状态空间模型（SSM / Mamba）	21
6.7	线性注意力	22
6.8	扩散式语言模型	23
7	量化：用更少比特存储	24
7.1	INT8 的麻烦：异常值	25
7.2	AWQ：激活感知的权重量化	25
7.3	蒸馏与剪枝：从大模型蒸馏到小模型	26
8	投机解码：用小模型猜，大模型验	26
8.1	流程：草稿 + 验证	27
8.2	拒绝采样：保证分布完全一致	27
8.3	实证：约 2 倍加速	28
9	服务层优化：批处理与内存管理	28
9.1	连续批处理	29
9.2	PagedAttention：KV cache 的虚拟内存	29
9.3	写时复制：共享前缀	30
10	总结与延伸	32
10.1	算术强度：贯穿全讲的统一线索	32
10.2	技术地图回顾	33
10.3	下节预告	33
10.4	配套阅读	33

1 为什么要严肃对待推理

Lecture 10 把镜头从训练转向推理（inference）。Percy 开篇强调一个工程现实：一个语言模型训练完成，只是成本故事的开始。真正烧钱的是把它部署起来、让成千上万的请求实时跑起来——这就是serving。本讲的全部内容都围绕一个核心问题展开：如何用尽量少的硬件、在尽量短的时间内，吐出尽量多的token。

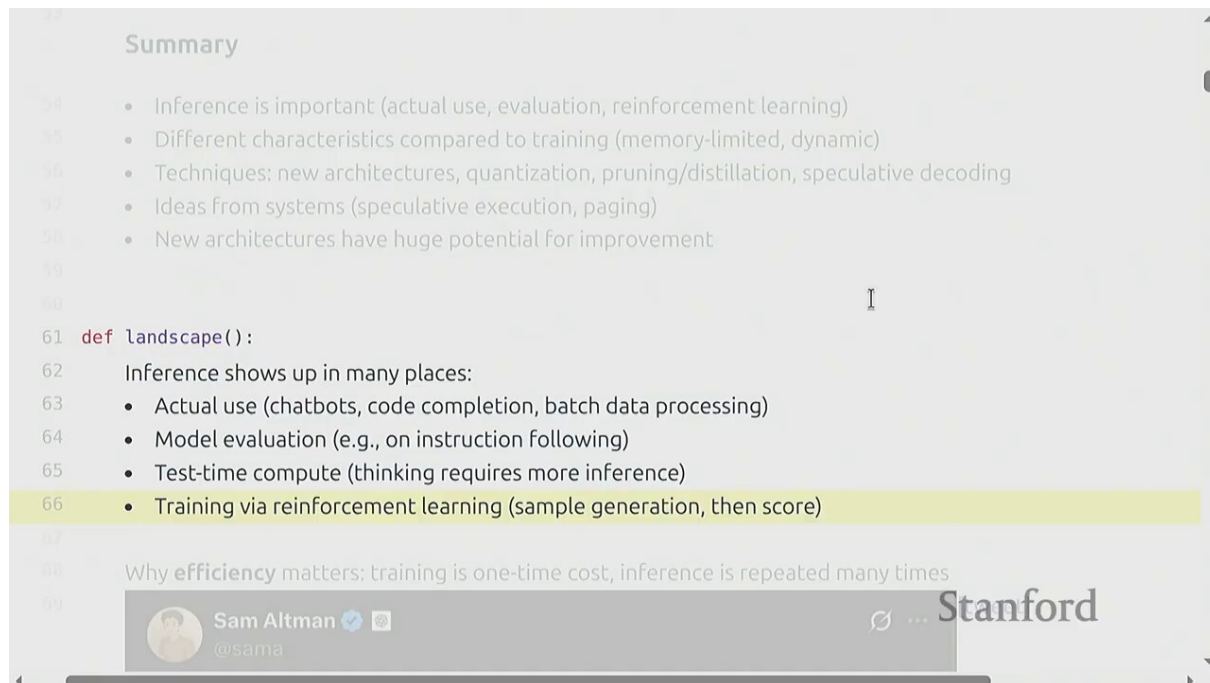


Figure 1: 从训练到推理：serving 才是成本的大头¹

为什么推理比训练更难优化

训练是一次大批次、长跑几周，瓶颈集中、容易吃满硬件；推理是高并发、变长请求、用户不等，瓶颈分散且随负载变化。训练时可以容忍一点浪费，推理时每一毫秒都直接关系用户体验和运营成本。

1.1 推理性能四个关键指标

讲清楚优化目标之前，先要明确衡量指标。Percy 给出了四个核心量：

¹视频画面时间区间：00:01:45 – 00:02:15。

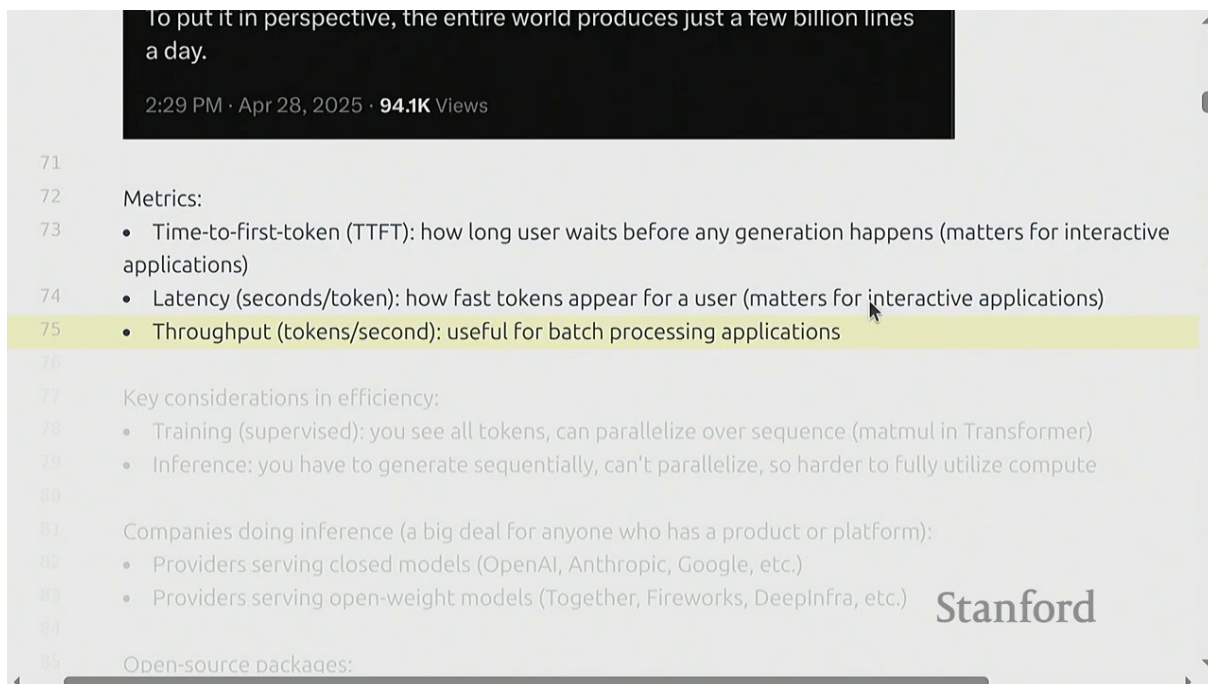


Figure 2: 推理服务的四大性能指标²

指标	符号	含义
首 token 延迟	TTFT	用户发送请求到收到第一个 token 的时间，决定”是否卡顿”
单 token 延迟	latency	生成每个 token 的平均时间，决定”打字快慢”
吞吐	throughput	单位时间（每秒）整个系统生成的 token 总数
成本	cost	每生成 100 万 token 需要多少美元

延迟与吞吐的对立统一

延迟（每个用户感受到的速度）和吞吐（系统整体效率）常常是此消彼长的。批量把多个请求凑一起跑，能拉高吞吐，但单个请求可能要等凑批，延迟反而升高。本讲后半段的连续批处理（continuous batching）就是在这两者之间找最优解。

1.2 统一符号定义

为了避免后面推导算术强度时符号混乱，先把全讲用到的记号固定下来：

²视频画面时间区间：00:03:45 – 00:04:15。

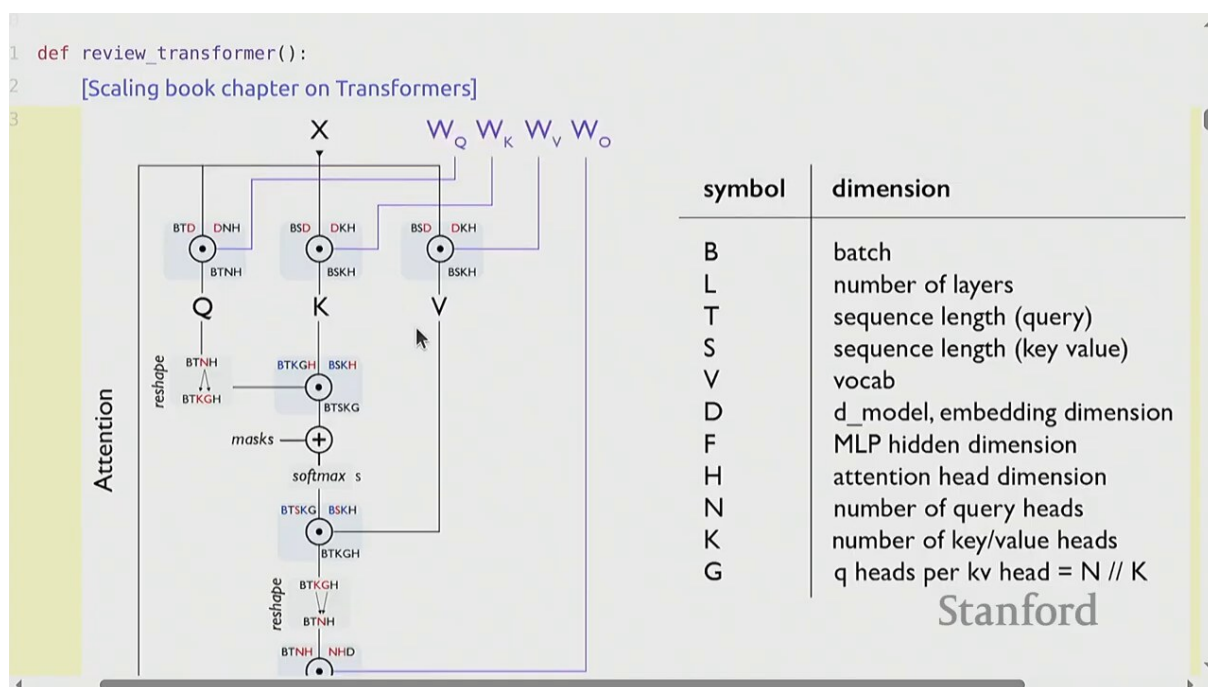


Figure 3: 贯穿全讲的统一符号³

符号	含义
B	批大小 (batch size) , 同时处理的请求数
L	序列长度, 一个请求包含的 token 数
T	解码阶段要生成的新 token 数
S	上下文 (prefill) 长度, 即 prompt 的 token 数
V	词表大小 (vocab size)
D	隐藏维度 (hidden dimension) , 模型主干的特征宽度
H	注意力头数

2 算术强度：推理优化的统一框架

本讲最重要的一个思维工具是算术强度 (arithmetic intensity) 。几乎所有推理优化技术, 都可以用这一个概念串起来。

2.1 矩阵乘法的 FLOPs 与字节

先看最基本的操作——矩阵乘法。这是 Transformer 里最贵的算子。对一个形状为 (B, D) 的输入乘以权重 (D, D) :

³视频画面时间区间: 00:06:20 – 00:06:50。

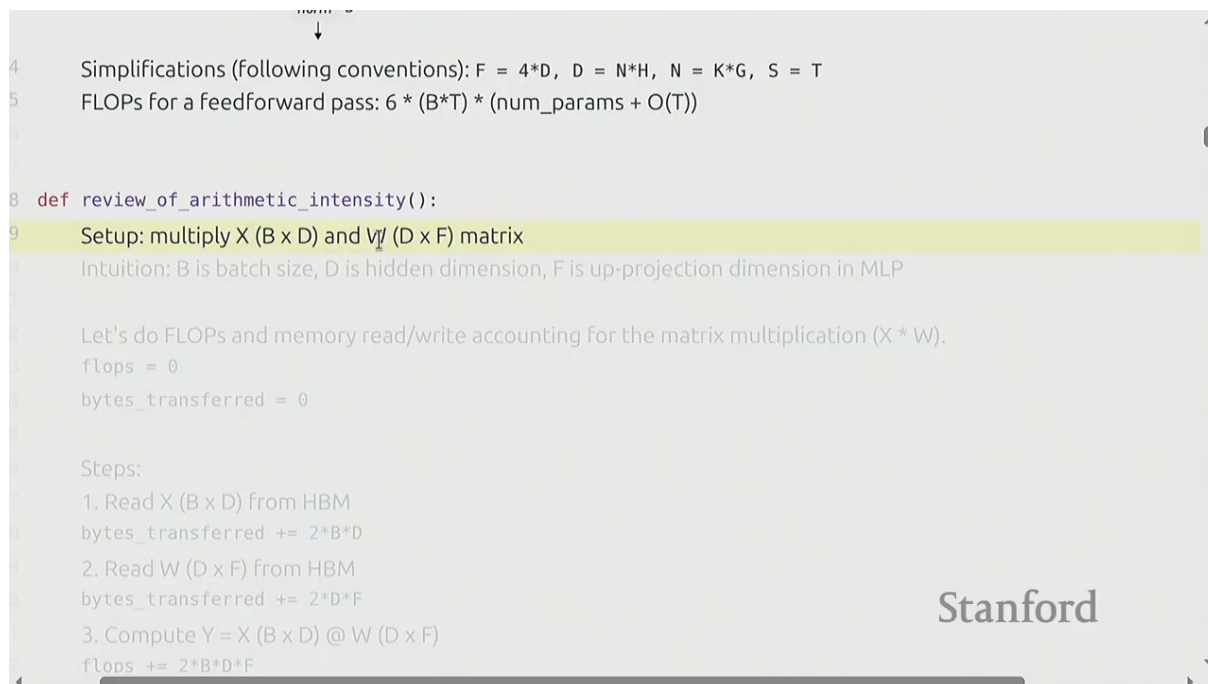


Figure 4: 矩阵乘法：算多少 FLOPs、读多少字节⁴

FLOPs 和数据搬运量分别是：

$$\text{FLOPs} = 2BD^2, \quad \text{bytes} = 2BD + 2D^2$$

- $2BD^2$ ：输出矩阵每个元素需要 $2D$ 次乘加（乘和加各算一次），共 BD 个输出元素
- $2BD$ ：读入激活矩阵的字节数（BF16 下每个元素 2 字节）
- $2D^2$ ：读入权重矩阵的字节数（权重每次都要重读）

2.2 算术强度的定义

算术强度的定义

算术强度 I 定义为每读一个字节做了多少次浮点运算：

$$I = \frac{\text{FLOPs}}{\text{bytes}}$$

它回答一个关键问题：这个算子到底是被算力卡住，还是被带宽卡住？

对上面的矩阵乘，当 $B \gg 1$ （大批量）时，权重 $2D^2$ 被很多请求摊薄，算术强度接近：

$$I \approx \frac{2BD^2}{2D^2} = B$$

而 $B = 1$ （单请求）时，激活和权重同量级：

$$I \approx \frac{2D^2}{2BD + 2D^2} = \frac{D}{B + D} \xrightarrow{B=1} \approx 1$$

关键直觉

$B = 1$ 时算术强度约为 1——做一次乘加几乎只读一个字节。这意味着单请求推理严重浪费算力，瓶颈完全在显存带宽上。

⁴视频画面时间区间：00:08:15 – 00:08:45。

2.3 H100 的 Roofline 阈值

为什么算术强度重要？因为每块 GPU 都有一条 **roofline** 曲线——它既受峰值算力（FLOPS）限制，也受峰值带宽（bytes/s）限制。两条线交点对应的算术强度，叫做转折点：

```
12 flops += 2*B*D*F
13
14 intensity = "B"
15
16
17 assert flops == 2*B*D*F
18 assert bytes_transferred == 2*B*D + 2*D*F + 2*B*F
19 Recall that arithmetic intensity is how much compute we do per byte transferred (want to be high).
20 intensity = (flops / bytes_transferred).simplify() # @inspect intensity
21
22 Assuming B is much less than D and F, then we can simplify:
23 intensity = intensity.subs(D, c*B).subs(F, c*B).limit(c, oo).simplify() # @inspect intensity
24 assert intensity == B
25
26 Accelerator intensity of H100:
27 flops_per_second = 989e12
28 memory_bandwidth = 3.35e12
29 accelerator_intensity = flops_per_second / memory_bandwidth # @inspect accelerator_intensity
30 assert round(accelerator_intensity) == 295
```

Figure 5: H100 的算力、带宽与算术强度阈值⁵

$$I^* = \frac{\text{FLOPS}}{\text{bandwidth}}$$

- FLOPS: GPU 每秒能做的浮点运算次数（H100 BF16 约 989 TFLOPS）
- bandwidth: GPU 每秒能从显存读出的字节数（H100 HBM3 约 3.35 TB/s）
- I^* : 算术强度阈值，H100 上 $\approx 989/3.35 \approx 295$

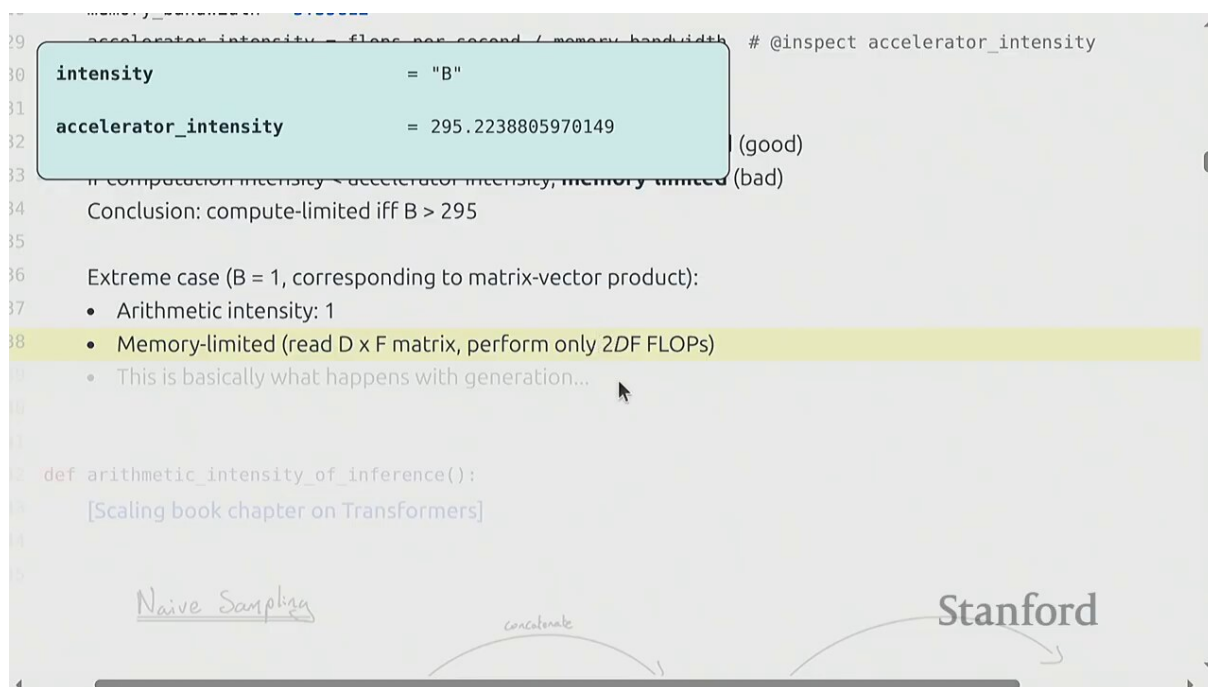
Roofline 的判读规则

若算子的 $I > I^*$ ，则算力受限（compute-bound）——加更多并发请求不会更快，因为已经把算力吃满了。若 $I < I^*$ ，则带宽受限（memory-bound）——光读权重就把时间耗光，算力闲置。H100 上这个阈值接近 300，是个非常高的门槛。

2.4 $B = 1$: 典型的带宽受限场景

把上面的数字代入单请求推理：

⁵视频画面时间区间：00:10:45 – 00:11:15。



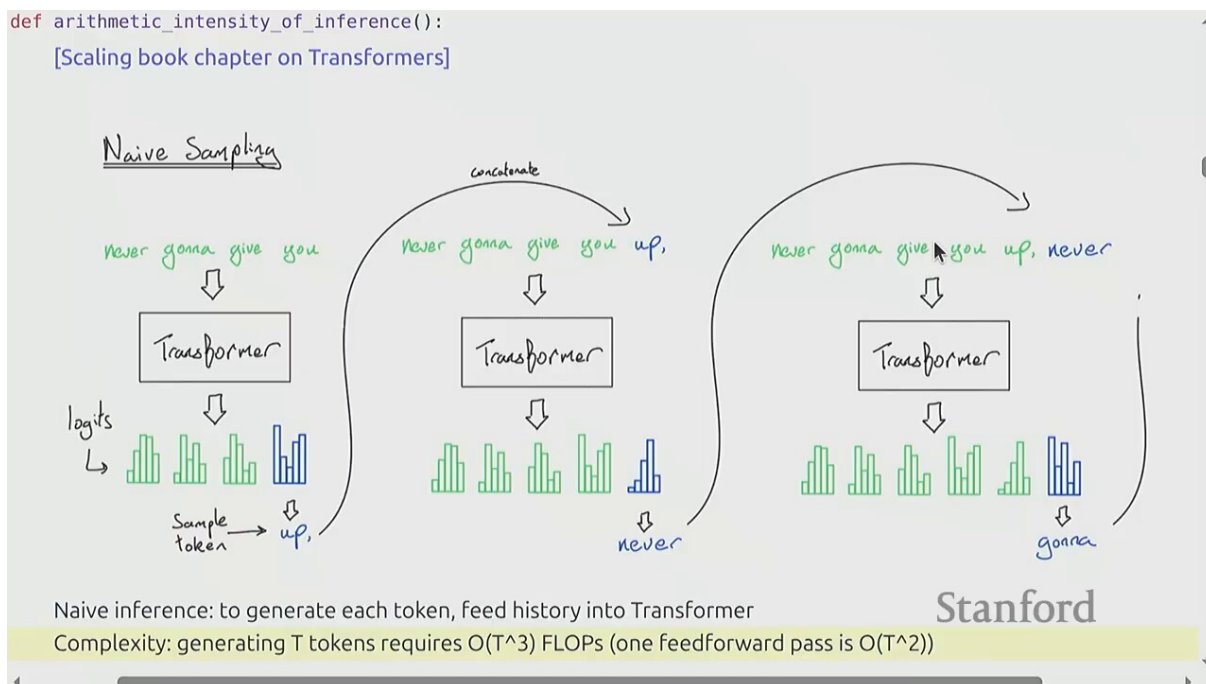


Figure 7: 朴素实现：第 t 步重算前 $t - 1$ 步的所有注意力⁷

朴素的 $O(N^2)$ 之痛

生成 T 个 token，朴素实现的总开销正比于 $1 + 2 + \dots + T = O(T^2)$ 。每个 token 的延迟随已生成长度线性增长——这根本没法用于长对话。

3.2 KV Cache 的核心思想

注意到注意力公式里，每个历史 token 只通过它的 K （键）和 V （值）参与计算，而且这两个量一旦算出来就不变——可以缓存下来复用：

⁷ 视频画面时间区间：00:15:15 – 00:15:45。

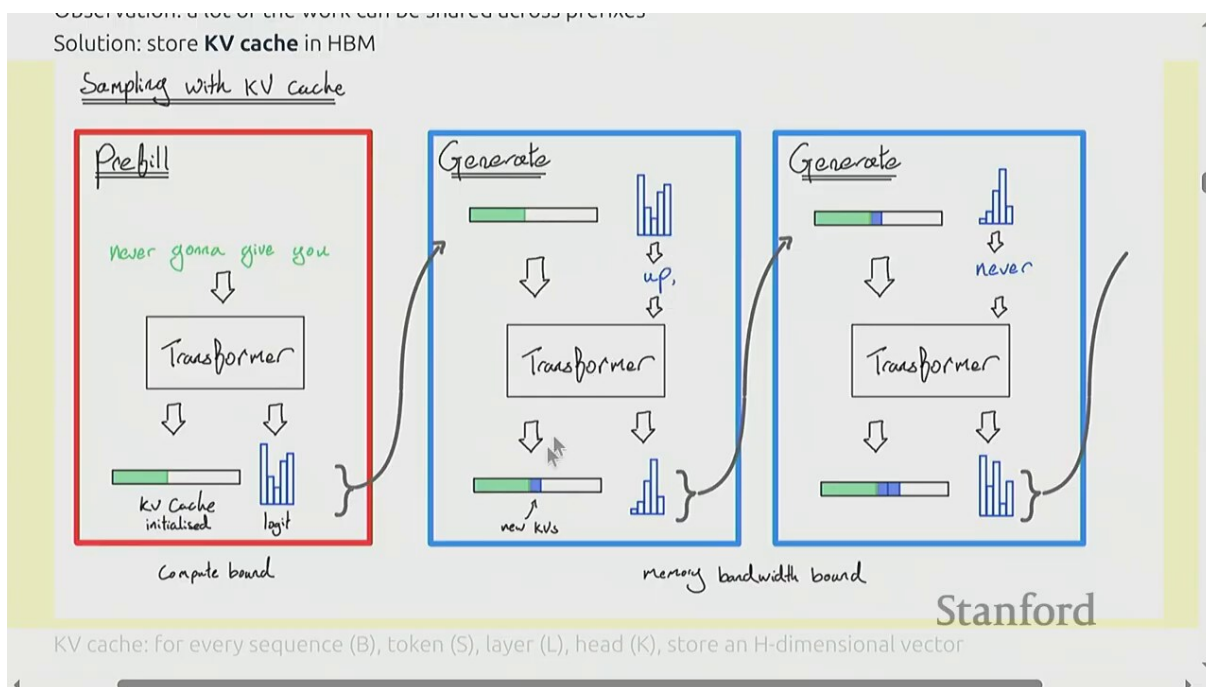


Figure 8: KV cache: 缓存历史 token 的键值，下一步只算新 token 的注意力⁸

KV cache 的本质

KV cache 用空间换时间：把每层每个头的 (K, V) 存下来，避免每步重算。代价是显存占用随序列长度线性增长——这恰恰是后面“批量请求会 OOM”的根本原因。

第 t 步，只需读入新 token 算出它的 q_t, k_t, v_t ，然后用 q_t 与缓存里所有的 K 做点积，再与 V 加权求和。每步注意力是 $O(t)$ 而非 $O(t^2)$ 。

3.3 推理的两个阶段：Prefill 与 Decode

有了 KV cache，推理自然分成两个节奏完全不同的阶段：

⁸视频画面时间区间：00:16:25 – 00:16:55。

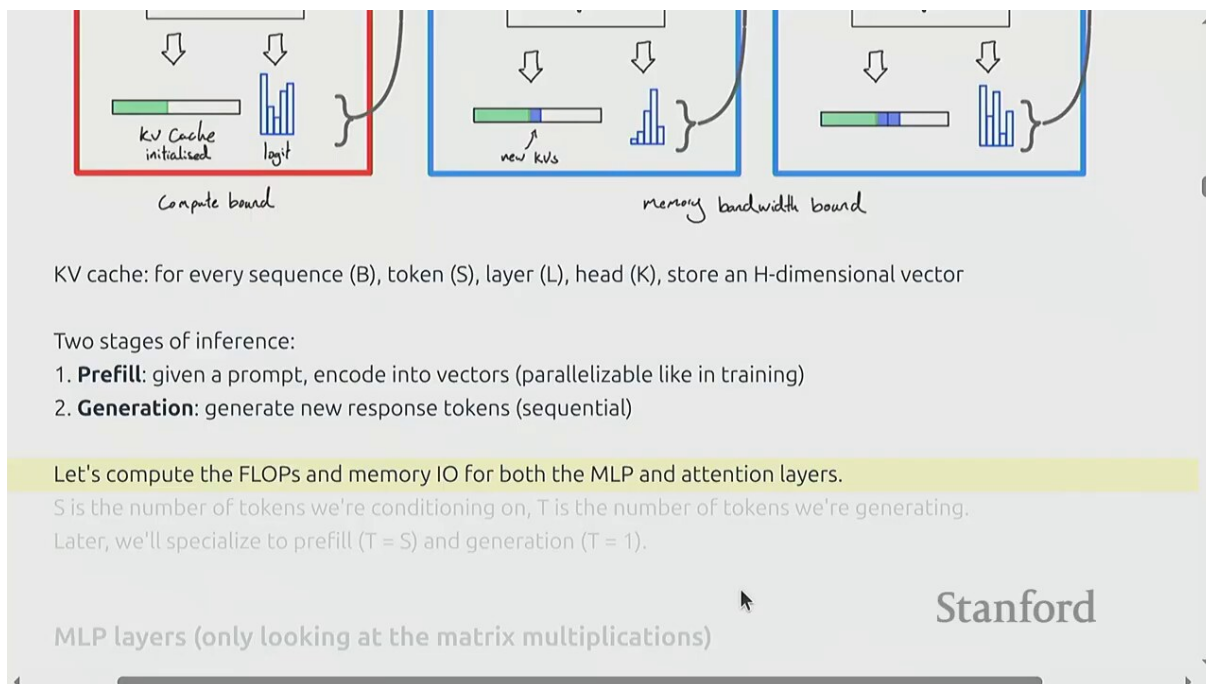


Figure 9: Prefill（预填充）与 Decode（解码）两个阶段⁹

	Prefill	Decode
做什么	一次性吃下整个 prompt，算出所有 token 的 KV 并缓存	逐个生成新 token，每步读 KV cache 算一个 token
批大小	prompt 长度 S （通常几百到几千）	B 个请求，每个只算 1 个 token
算术强度	高 ($\approx S$)	低 (≈ 1)
瓶颈	算力 (compute-bound)	带宽 (memory-bound)

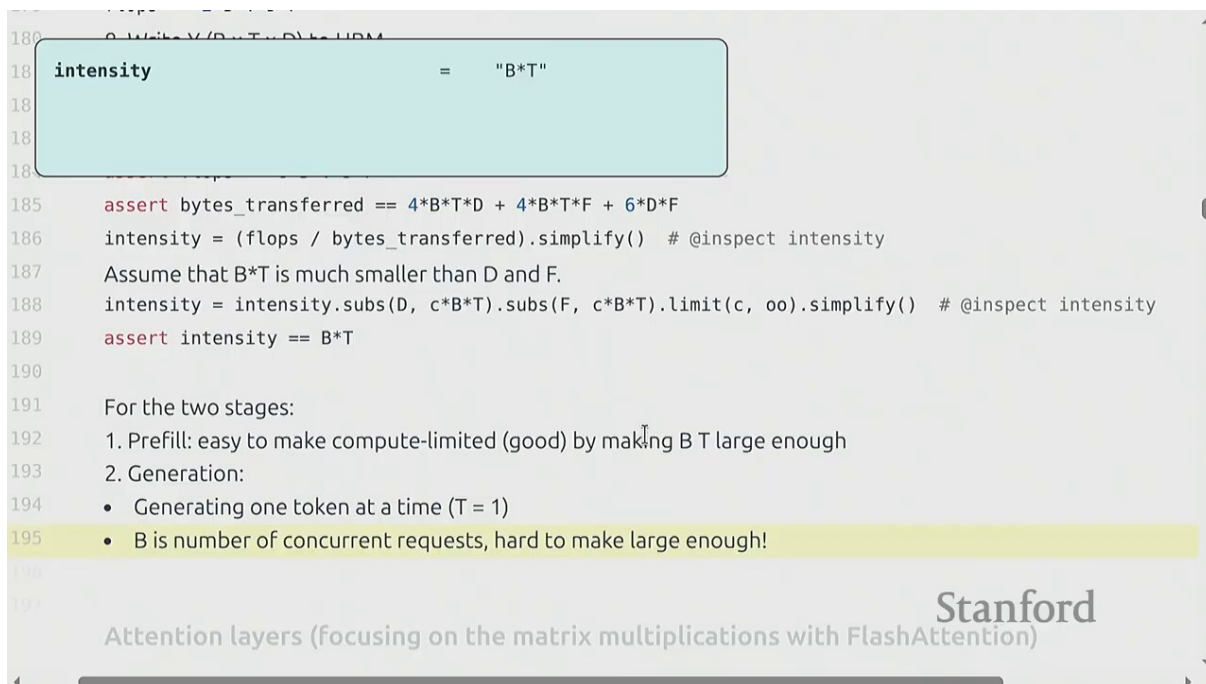


Figure 10: Prefill 阶段：大批量矩阵乘，算术强度 $\approx S$ ¹⁰

⁹视频画面时间区间：00:17:45 – 00:18:15。

¹⁰视频画面时间区间：00:21:15 – 00:21:45。

Prefill 是算力受限

Prefill 把 prompt 当作一个 $B = S$ 的大批次做矩阵乘，算术强度 $\approx S$ （几百到几千），远超 H100 阈值 300。这一阶段把算力吃满，是 **compute-bound**。优化方向是张量并行、更好的 kernel。

4 注意力的算术强度推导

Percy 接下来用一个漂亮的小推导，回答了一个看似细节、实则关键的问题：解码阶段的注意力本身，算术强度到底是多少？

4.1 Decode 阶段的注意力： $\sim ST/(S + T)$

```
198     flops = 0
    intensity = "S*T/(S + T)"
203     2. Compute A = Q (B x T x D) @ K (B x S x D)
204     flops += 2*B*S*T*D
205     3. Compute Y = A (B x S x T x K x G) @ V (B x S x K x H)
206     flops += 2*B*S*T*D
207     4. Write Y (B x T x D) to HBM
208     bytes_transferred += 2*B*T*D
209
210     assert flops == 4*B*S*T*D
211     assert bytes_transferred == 4*B*S*D + 4*B*T*D
212     intensity = (flops / bytes_transferred).simplify() # @inspect intensity
213     assert intensity == S*T / (S + T)
214
215     For the two stages:
216     1. Prefill: T = S
217     prefill_intensity = intensity.subs(T, S).simplify() # @inspect prefill_intensity
```

Figure 11: 解码注意力的算术强度 $\sim ST/(S + T)$ ¹¹

设上下文长度 S 、生成长度 T 。每步解码，注意力的 FLOPs 来自 q 与所有 K 的点积，以及 softmax 后与 V 的加权求和：

$$\text{FLOPs}_{\text{attn}} \sim B \cdot (S + T) \cdot D, \quad \text{bytes}_{\text{attn}} \sim B \cdot (S + T) \cdot D$$

- $B \cdot (S + T) \cdot D$ ：要读的 KV cache 字节数（每个历史 token 的 K 和 V）
- 注意力的 FLOPs 与读的字节同量级，因为每个 KV 元素只参与 $O(1)$ 次乘加

把整个生成过程平均下来，每个 token 的有效算术强度约为：

$$I_{\text{attn}} \sim \frac{ST}{S + T}$$

4.2 当 $T \gg S$ ：算术强度趋于 1

当生成很长（ $T \gg S$ ，即多轮长对话），分母里 S 可忽略：

¹¹ 视频画面时间区间：00:23:55 – 00:24:25。

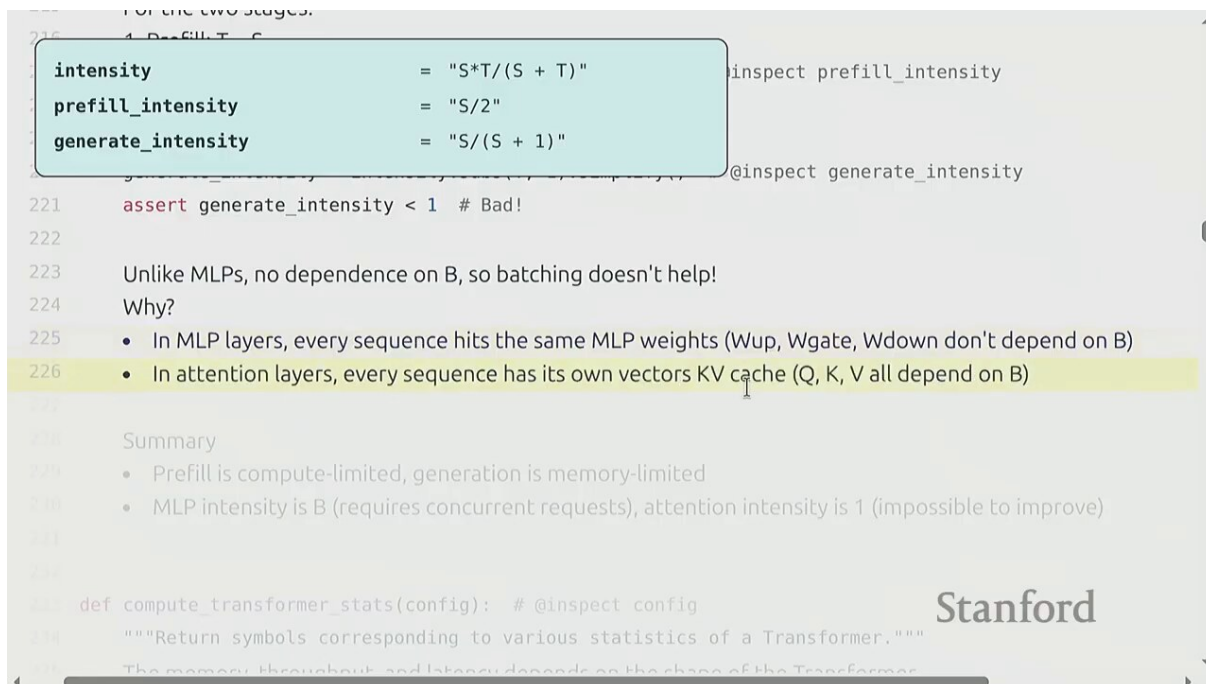


Figure 12: $T \gg S$ 时：解码注意力强度 $\rightarrow S$ （仍远低于阈值）¹²

$$I_{\text{attn}} \xrightarrow{T \gg S} S$$

注意力的致命弱点

即便考虑上下文长度 S ，注意力的算术强度也只有 S 量级（几千），而每个 token 的读写量（KV cache 体积）随序列长度线性膨胀。注意力层的瓶颈永远是带宽，而且越长的上下文越严重。这就是为什么后面所有架构改造（GQA、MLA、SSM）都在攻击 KV cache 的体积。

MLP 与注意力的对比

对比之下，MLP 层（前馈网络）每个 token 要乘两个大权重矩阵，算术强度 $\approx B$ ——所以加大批量能直接让 MLP 变得算力受限、提高 GPU 利用率。但注意力的强度与 B 无关，加批量救不了它。这是本讲最重要的不对称性。

5 Llama2 手算：把公式落到具体数字

理论讲完，Percy 用 Llama2 的真实配置做了一次“餐巾纸算术”（napkin math），让所有公式落地：

¹²视频画面时间区间：00:25:35 – 00:26:05。

```

53
54 # Substitute
55 num_params = num_params.subs(config).simplify() # @inspect num_params
56 memory = memory.subs(config).simplify() # @inspect memory
57 latency = latency.subs(config).simplify() # @inspect latency
58 throughput = throughput.subs(config).simplify() # @inspect throughput
59
60 return num_params, memory, latency, throughput
61
62 def llama2_13b_config(args={}):
63     return {S: 1024, D: 5120, F: 13824, N: 40, K: 40, H: 128, L: 40, V: 32000, memory_bandwidth: 3.35e12,
64
65 def throughput_and_latency():
66     So we have shown that inference is memory-limited.
67     Let us now compute the theoretical maximum latency and throughput of a single request.
68     Assumption: can overlap compute and communication perfectly and ignore various types of overhead.
69
70     Instantiate latency and throughput for Llama 2 13B on an H100:
71     config = llama2_13b_config()
72     num_params, memory, latency, throughput = compute_transformer_state(config)

```

Figure 13: Llama2 配置与单卡推理性能估算¹³

Llama2-7B 的关键参数：层数 $L = 32$ ，隐藏维度 $D = 4096$ ，头数 $H = 32$ ，每个权重大约 70 亿参数。单次前向（生成 1 个 token）需要：

$$\text{FLOPs}_{\text{token}} \approx 2 \cdot N_{\text{params}} = 2 \times 7\text{B} = 14 \text{ GFLOPs}$$

而要把权重全读一遍（BF16）：

$$\text{bytes}_{\text{token}} \approx 2 \cdot N_{\text{params}} = 14 \text{ GB}$$

- N_{params} ：模型参数量（Llama2-7B 约 70 亿）
- $2 \cdot N_{\text{params}}$ ：每个 token 要算的 FLOPs（乘加各算一次），BF16 下也是要读的字节数

单 token 时间的下界

单请求解码时算术强度 ≈ 1 ，所以每秒能生成的 token 数被带宽卡死：

$$\text{tokens/s} \approx \frac{\text{bandwidth}}{\text{bytes}_{\text{token}}} = \frac{3.35 \text{ TB/s}}{14 \text{ GB}} \approx 240 \text{ tokens/s}$$

这是理论下界——单卡单请求，Llama2-7B 最多每秒吐约 240 个 token（实际因 kernel 开销会更低）。

5.1 批量：把吞吐拉上去

单请求只用了带宽、算力闲置。把多个请求凑成一拨跑，权重只读一次、服务多个请求，算术强度就上去了：

¹³视频画面时间区间：00:28:45 – 00:29:15。


```

return {S: 1024, D: 5120, F: 13824, N: 40, K: 40, H: 128, L: 40, V: 32000, memory_bandwidth: 3.35e12, **}

def throughput_and_latency():
    So we have shown that inference is memory-limited.
    Let us now compute the theoretical maximum latency and throughput of a single request
    Assumption: can overlap compute and communication

    Instantiate latency and throughput for LLaMA2
    config = llama2_13b_config()
    num_params, memory, latency, throughput = compute_transformer_stats(config)
    assert num_params == 13_015_449_600 # Sanity check

    If we use a batch size of 1:
    bs1_memory = memory.subs(B, 1).simplify() # @inspect bs1_memory
    bs1_latency = latency.subs(B, 1).simplify() # @inspect bs1_latency
    bs1_throughput = throughput.subs(B, 1).simplify() # @inspect bs1_throughput

    If we use a batch size of 64 (worse latency, better throughput):
    bs64_memory = memory.subs(B, 64).simplify() # @inspect bs64_memory
    bs64_latency = latency.subs(B, 64).simplify() # @inspect bs64_latency

```

bs1_memory = 26,869,760,000

bs1_latency = 0.008020823880597015

Stanford

Figure 14: 加大批量：吞吐上升、单 token 延迟不变（带宽足够宽）¹⁴

批量的甜区

在带宽受限区，加大批量 B 能让吞吐线性增长（因为算力还没饱和），同时单 token 延迟几乎不变（带宽够宽，多读点 KV 不碍事）。这是 serving 的“免费午餐”——直到两个限制出现：（1）KV cache 把显存撑爆；（2）算力饱和，开始互相挤。

¹⁴视频画面时间区间：00:32:15 – 00:32:45。

5.2 $B = 256$: KV cache 把显存撑爆

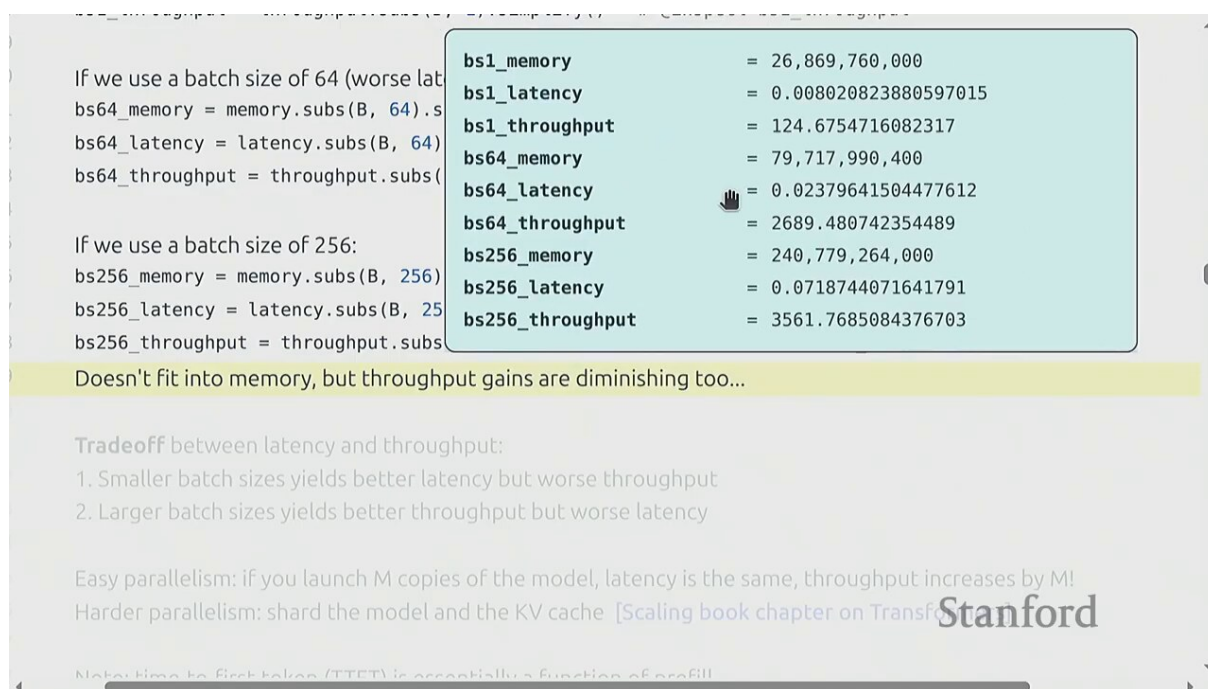


Figure 15: $B = 256$ 时: KV cache 占满显存, OOM¹⁵

KV cache 的显存占用为:

$$\text{KV memory} = 2 \cdot L \cdot H \cdot D_{\text{head}} \cdot (S + T) \cdot B \cdot 2 \text{ bytes}$$

- 2: K 和 V 两个张量
- L : 层数, H : 头数, $D_{\text{head}} = D/H$: 每头维度
- $(S + T)$: 每条请求的序列总长
- B : 批大小, 最后乘 2 bytes 是 BF16

Llama2-7B 上, $B = 256$ 、 $S = T = 2048$ 时, KV cache 就要吃掉几十 GB, 远超单卡显存。显存成了批量的天花板。

5.3 复制与张量并行: 跨卡扩展

单卡装不下怎么办? 两条路:

¹⁵视频画面时间区间: 00:33:35 – 00:34:05。

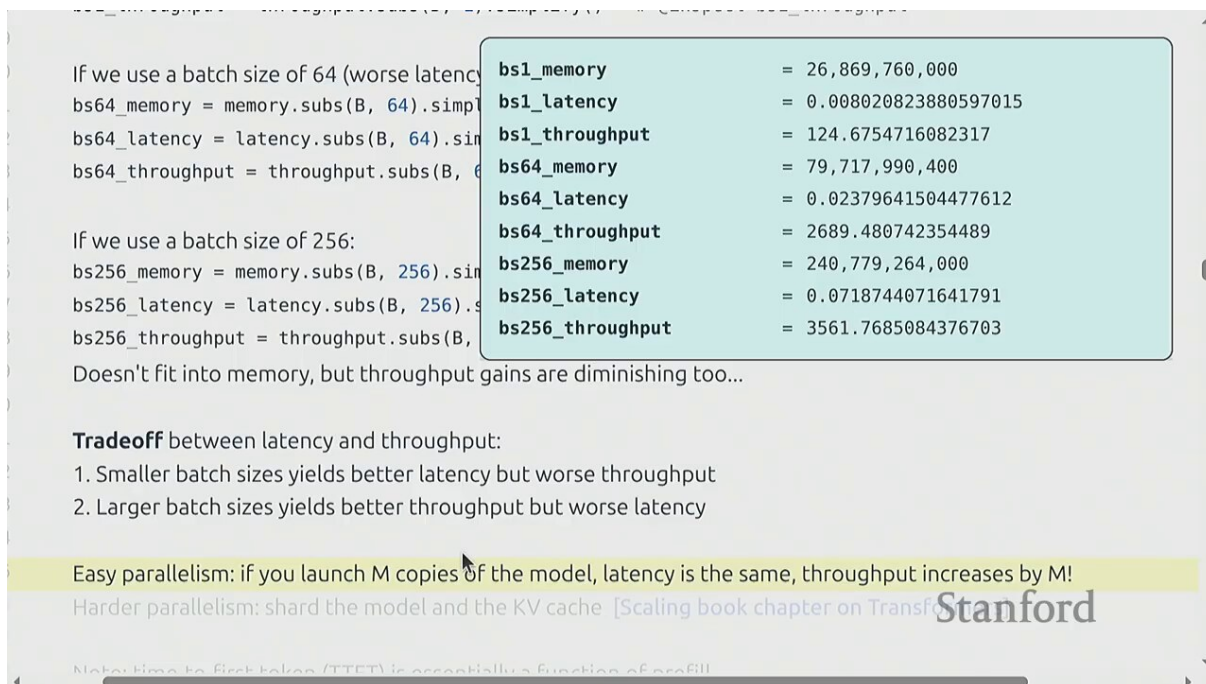


Figure 16: 复制（多份模型接不同请求）vs 张量并行（一切模型分到多卡）¹⁶

	复制 (replication)	张量并行 (TP)
做法	同一模型拷贝几份，每份独立接不同请求	一个模型切成几块，每块放一张卡，请求同时跑
适用	模型小、单卡装得下	模型大、单卡装不下
吞吐	线性扩展（最理想）	受通信开销影响
延迟	不变	略升（要跨卡同步）

复制的隐藏代价

复制虽然吞吐线性扩展，但每份拷贝都要存完整的权重——显存利用率低。当请求量小时，多份拷贝可能各跑各的低批量，反而比一份统一跑大批量更亏。这就是为什么有“selective batching”——只在能凑批量时合并，否则拆开。

6 改造架构：降低推理的固有成本

接下来 Percy 转向一个更深层的问题：上面的分析都假设了标准 Transformer 架构。如果改架构，能不能从根上把推理成本降下来？

¹⁶视频画面时间区间：00:34:25 – 00:34:55。



Figure 17: 降低推理成本的架构改造路线图¹⁷

6.1 标准 MHA: KV cache 的代价

先看标准的多头注意力 (Multi-Head Attention)。每个头 h 都有自己独立的 K_h, V_h :

2. Larger batch sizes yields better throughput but worse latency

294

295 Easy parallelism: if you launch M copies of the model, latency is the same, throughput increases by M!

296 Harder parallelism: shard the model and the KV cache [Scaling book chapter on Transformers]

297

298 Note: time-to-first-token (TTFT) is essentially a function of prefill

299 Use smaller batch sizes during prefill for faster TTFT

300 Use larger batch sizes during generation to improve throughput

301

302

303 `def reduce_kv_cache_size():`

304 Recall that memory is the bottleneck for inference.

305 So let's try to reduce the size of the KV cache

306 ...but make sure we don't lose too much accuracy.

307

308

309

Grouped-query attention (GQA)

[Ainslie+ 2023]

Stanford

Multi-head Grouped-query Multi-query

Figure 18: 标准 MHA: H 个头各存自己的 KV¹⁸

KV cache 总体积正比于 $H \cdot D_{\text{head}}$ 。头数越多, KV cache 越胖, 带宽压力越大。

¹⁷ 视频画面时间区间: 00:36:45 – 00:37:15。

¹⁸ 视频画面时间区间: 00:38:45 – 00:39:15。

6.2 MQA: 所有头共享一组 KV

Multi-Query Attention (MQA) 做了一个激进的简化: 所有查询头共享同一组 K, V :

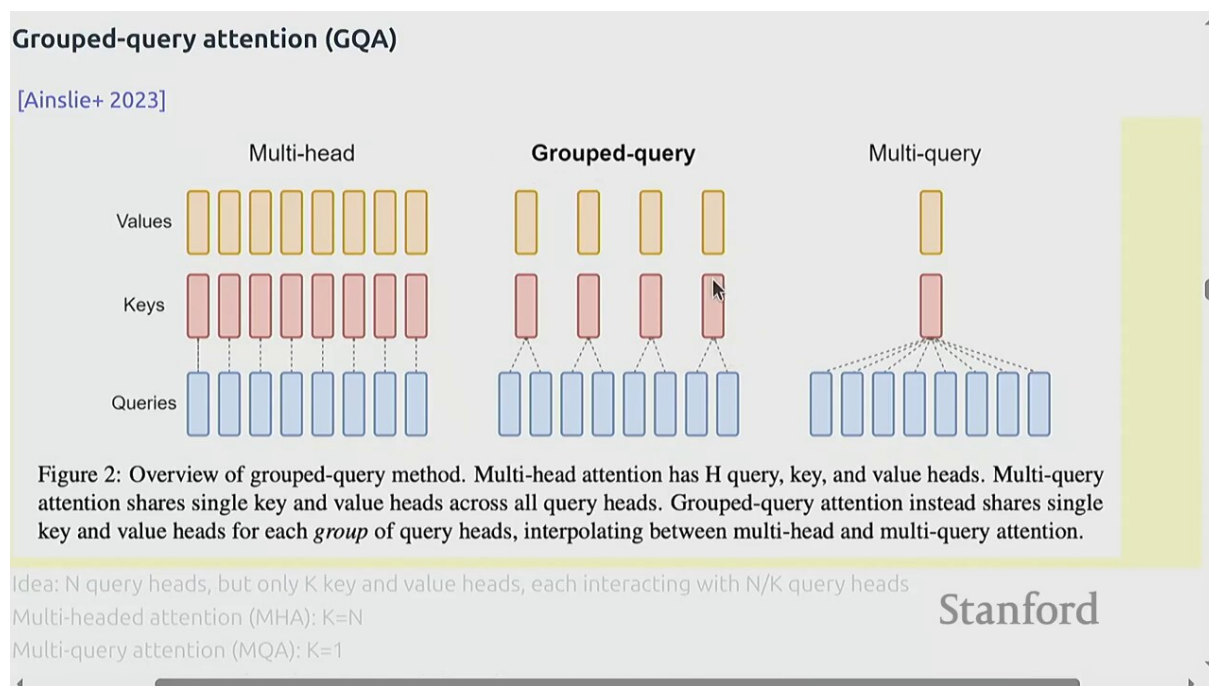


Figure 19: MQA: 所有查询头共享一组 KV, 体积降至 $1/H$ ¹⁹

MQA 的取舍

KV cache 体积直接降到 $1/H$ (如 $H = 32$ 就是 32 倍节省), 带宽和显存压力大幅下降。代价是: 所有查询头看的是同一份 KV, 表达能力下降, 模型质量略降。在推理速度远比“一点点精度”重要的场景, 这笔交易非常划算。

6.3 GQA: 折中方案

Llama2-70B 等采用 **Grouped-Query Attention (GQA)**, 在 MHA 和 MQA 之间取折中: 把 H 个查询头分成 G 组, 每组共享一组 KV:

¹⁹视频画面时间区间: 00:40:15 – 00:40:45。

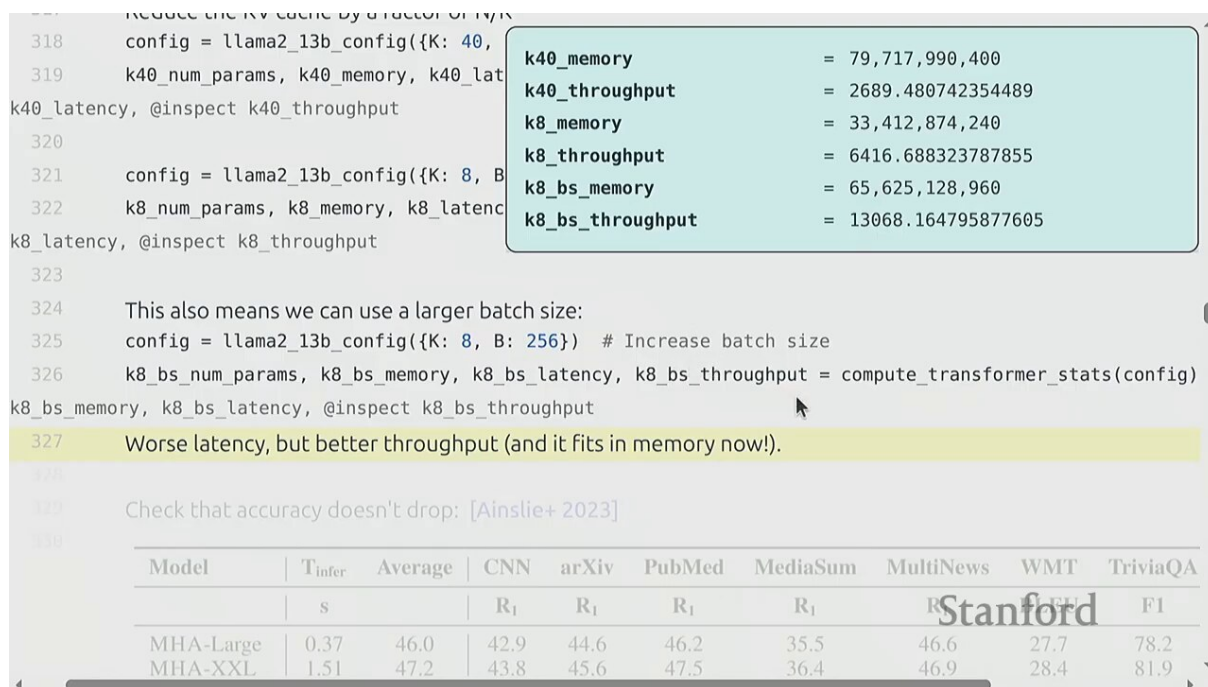


Figure 20: GQA: H 个查询头分 G 组，组内共享 KV²⁰

- $G = H$: 退化为标准 MHA（每组只有一个头，各自独立）
- $G = 1$: 退化为 MQA（所有头一组）
- $1 < G < H$: KV cache 体积是 MHA 的 G/H ，质量损失比 MQA 小

为什么 GQA 是主流

GQA 提供了一个平滑旋钮——在显存/带宽节省和模型质量之间连续调节。Llama2-70B 用 $G = 8$ ($H = 64$)，KV cache 省 8 倍但质量几乎不损。这种“几乎免费”的优化让 GQA 成了新一代大模型的标配。

6.4 MLA: DeepSeek 的低秩压缩

Multi-head Latent Attention (MLA, DeepSeek 提出) 更进一步：把 KV 压缩到一个低秩潜在向量，只在需要时才展开成完整的 K、V：

²⁰ 视频画面时间区间：00:42:15 – 00:42:45。

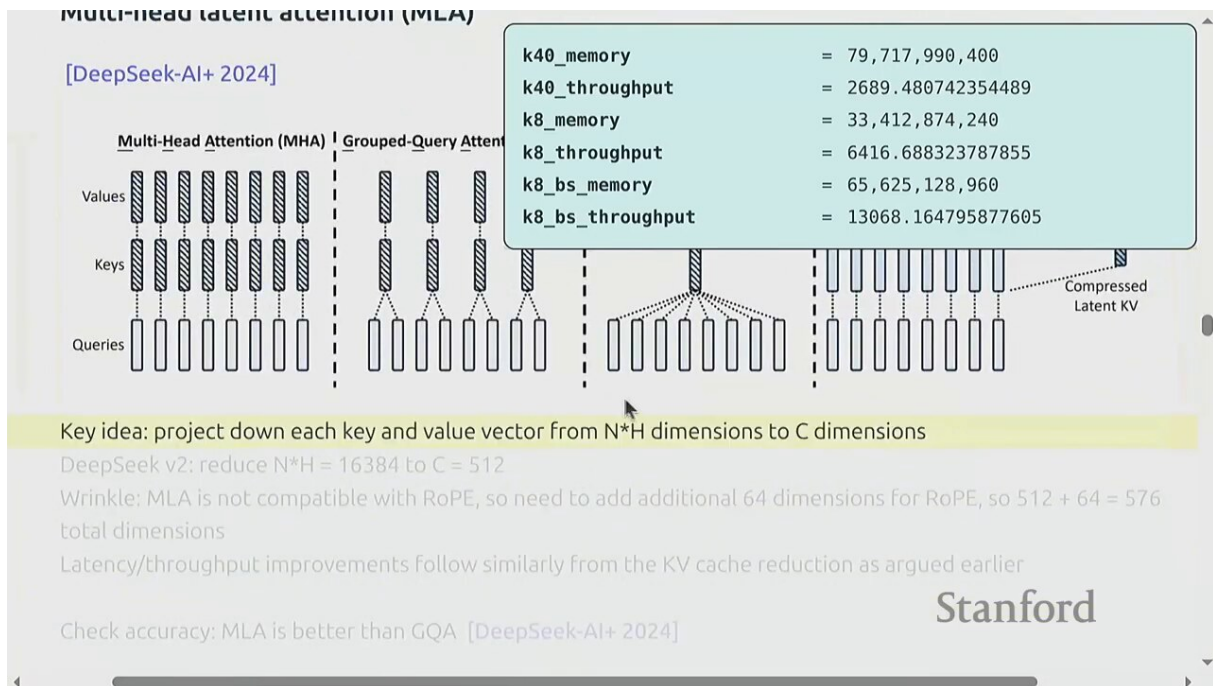


Figure 21: MLA (DeepSeek) : KV 先压到低秩潜在向量 c , 再投影展开²¹

$$K = W_K c, \quad V = W_V c, \quad c = W_c x$$

- x : 输入隐藏状态 (高维 D)
- c : 压缩后的潜在向量 (低维 $d_c \ll D \cdot H$)
- W_K, W_V : 把潜在向量投影回完整 K 、 V 的矩阵

MLA 的精妙

MLA 只缓存压缩向量 c , 而不是完整的 K 、 V 。需要时再用 W_K, W_V 投影出来。这让 KV cache 体积再降一个数量级, 同时保留了多头的表达能力。代价是额外的投影计算, 但在带宽受限的解码阶段, 省下的带宽远比多算的 FLOPs 划算。

6.5 局部注意力与跨层共享

除了改 KV 的组织方式, 还可以减少每个 token 需要存的 KV 数量:

²¹ 视频画面时间区间: 00:44:15 – 00:44:45。

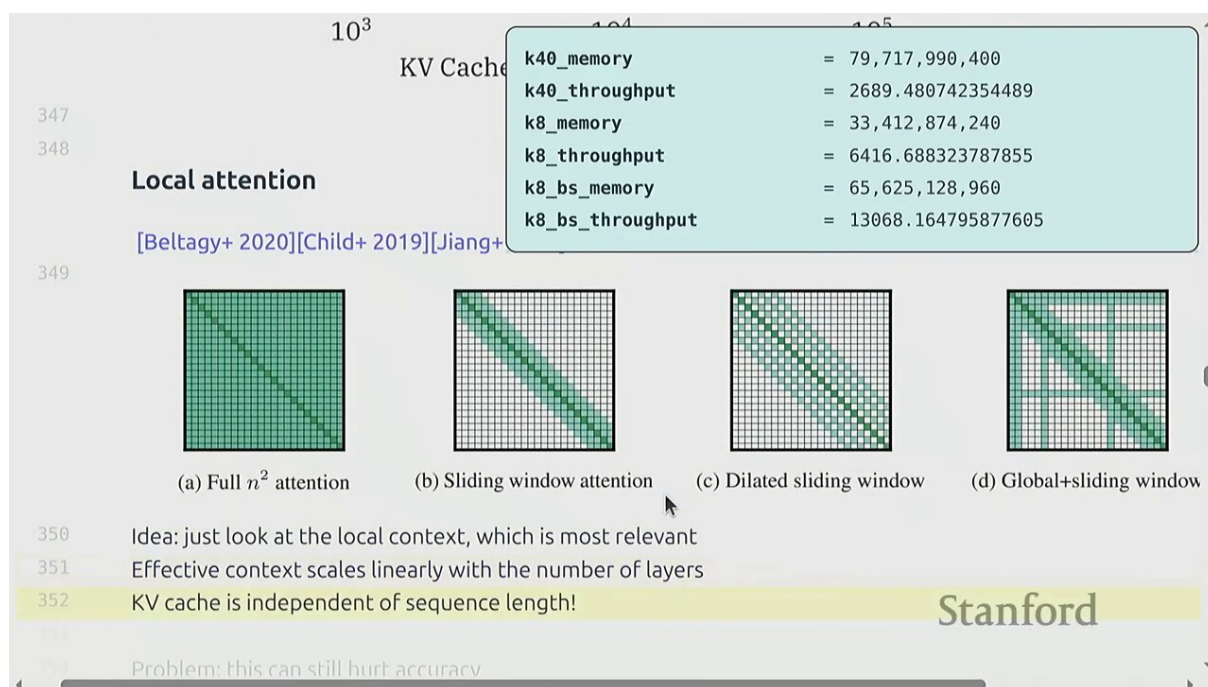


Figure 22: 局部/滑动窗口注意力 + CLA 跨层共享 KV²²

技术	做法与代价
滑动窗口注意力	每个 token 只看最近 w 个 token, KV cache 大小恒定 $O(w)$ 。代价: 丢失远距离依赖
跨层注意力 (CLA)	相邻层共享同一份 KV, 少算几组 K、V。代价: 层间耦合, 可能影响表达力

6.6 状态空间模型 (SSM / Mamba)

更激进的思路: 换掉注意力本身。状态空间模型用一个固定大小的隐状态 h_t 替代变长的 KV cache:

²²视频画面时间区间: 00:48:45 – 00:49:15。

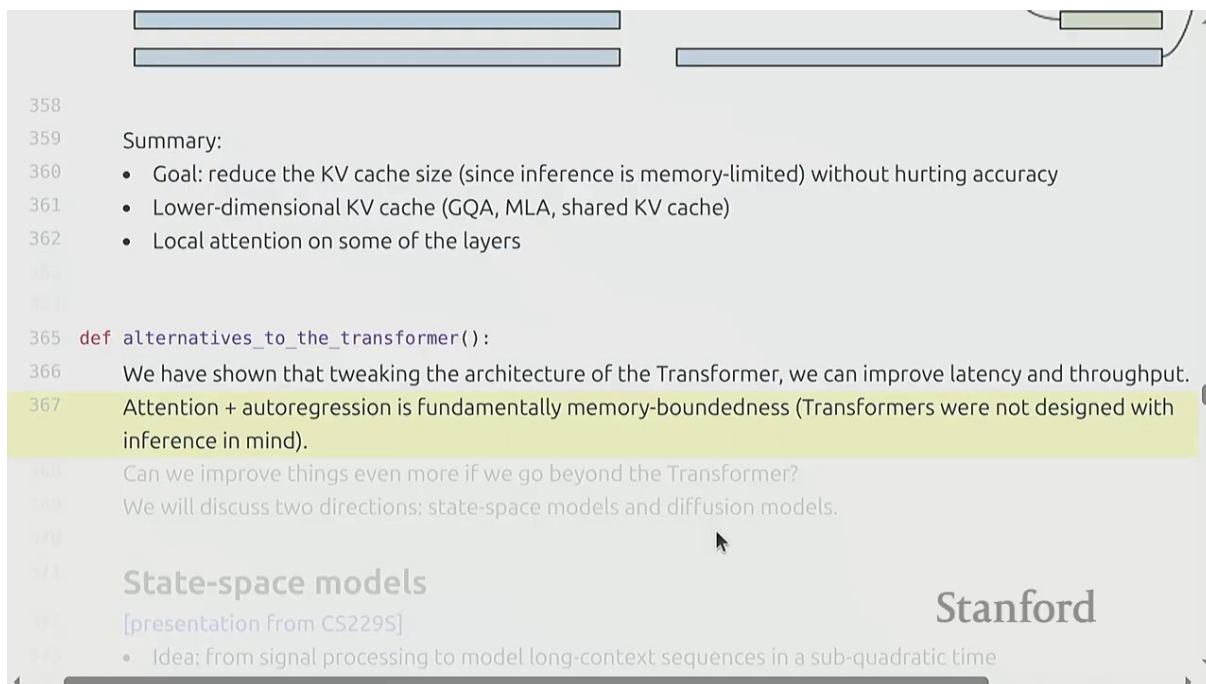


Figure 23: SSM / Mamba: 固定大小隐状态替代变长 KV cache²³

$$h_t = A h_{t-1} + B x_t, \quad y_t = C h_t$$

- h_t : 时刻 t 的隐状态（固定维度，不随序列增长）
- A, B, C : 状态转移、输入、输出矩阵（可学习的参数）
- x_t : 当前 token 的输入特征

SSM 的推理优势

SSM 的隐状态大小恒定，推理时每步的计算量和显存占用与序列长度无关——从根本上解决了注意力的 $O(N)$ 增长问题。训练时也可以做成线性复杂度（不再 $O(N^2)$ ）。Mamba 等模型让 SSM 在质量上首次追平 Transformer。

6.7 线性注意力

线性注意力是注意力的“软”替代：用核函数 ϕ 把 Q, K, V 改写，让复杂度从 $O(N^2)$ 降到 $O(N)$ ：

²³视频画面时间区间：00:53:15 – 00:53:45。

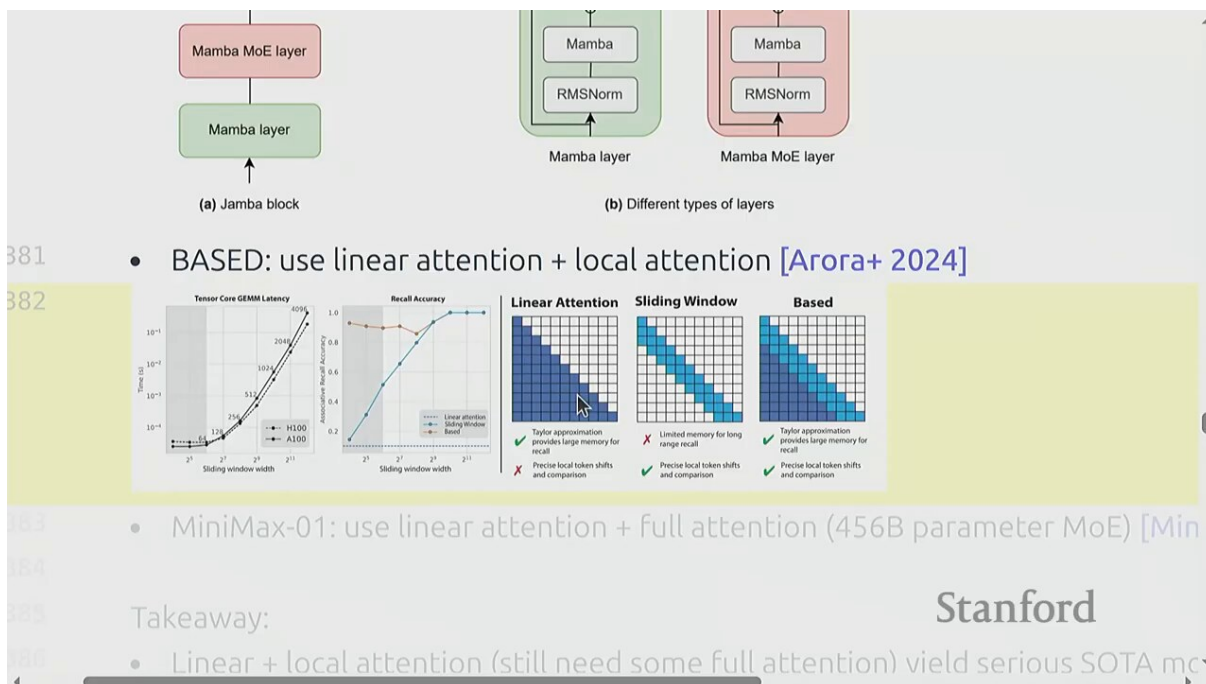


Figure 24: 线性注意力：用核函数把 $O(N^2)$ 改写为 $O(N)$ ²⁴

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \rightarrow \phi(Q)(\phi(K)^\top V)$$

- 左式：标准注意力，复杂度 $O(N^2)$ （每对 token 都要点积）
- 右式：线性注意力，先把 $\phi(K)^\top V$ 算成一个固定大小的“记忆矩阵”，再乘 $\phi(Q)$ ，复杂度 $O(N)$
- ϕ ：核函数，把 Q, K 映射到某个特征空间（例如 $\phi(x) = \text{elu}(x) + 1$ ）

线性注意力的现代复兴

MiniMax 456B 等新模型大量采用线性注意力变体——大部分层用线性注意力（保持 $O(N)$ ），只在关键层保留标准注意力（保留表达力）。这种混合架构是当前试图兼得速度和质量的主流方向。

6.8 扩散式语言模型

最后还提到了一类非自回归的方案——扩散模型（diffusion）。它不再“一个一个 token 往外吐”，而是从噪声出发、并行地去噪出一整段文本：

²⁴视频画面时间区间：00:58:15 – 00:58:45。

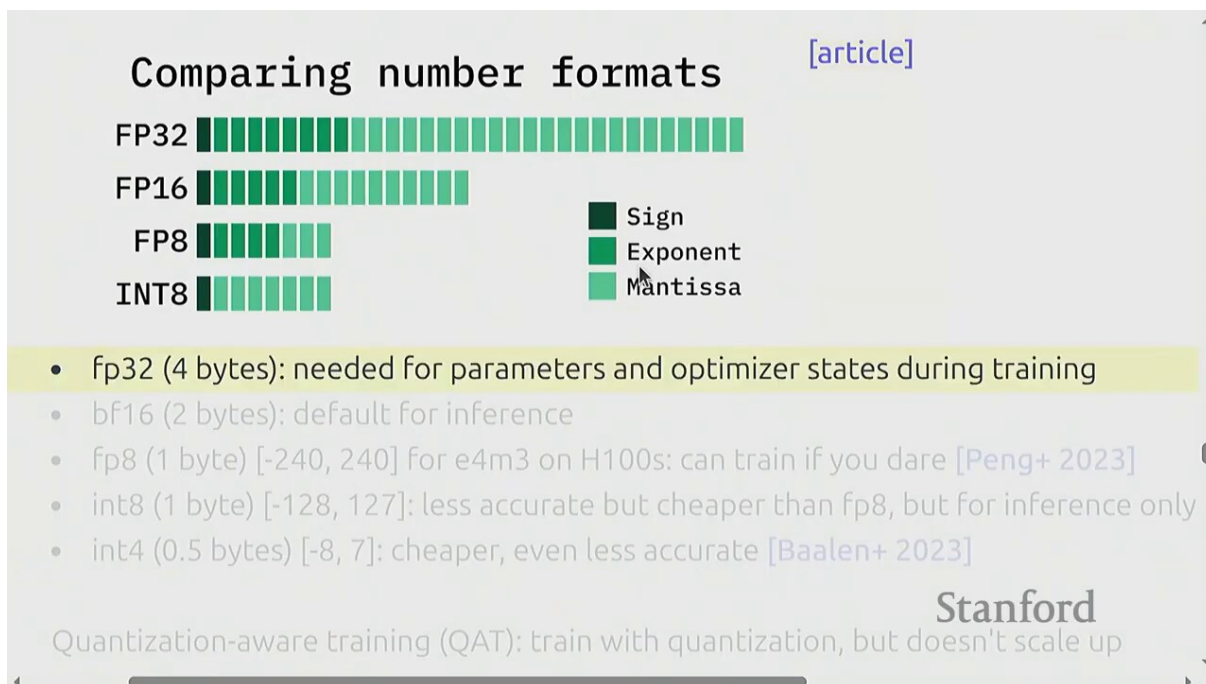


Figure 25: 从训练完的模型出发：用更少比特存储权重与激活（量化）²⁵

扩散 LLM 的潜力与挑战

扩散式生成把“自回归的串行”变成“并行的去噪”，理论上吞吐可以大幅提升。Sohl-Dickboy 等的早期工作和近期的图像生成成功给了信心。但语言是离散的，把连续去噪搬到离散 token 上“相当 tricky”，目前质量还没追上自回归。这是一个值得关注的开放方向。

7 量化：用更少比特存储

架构改造是“从根上动刀”，量化（quantization）则是“不动架构、只动精度”——用更少的比特来存权重和激活。

²⁵视频画面时间区间：01:05:15 – 01:05:45。

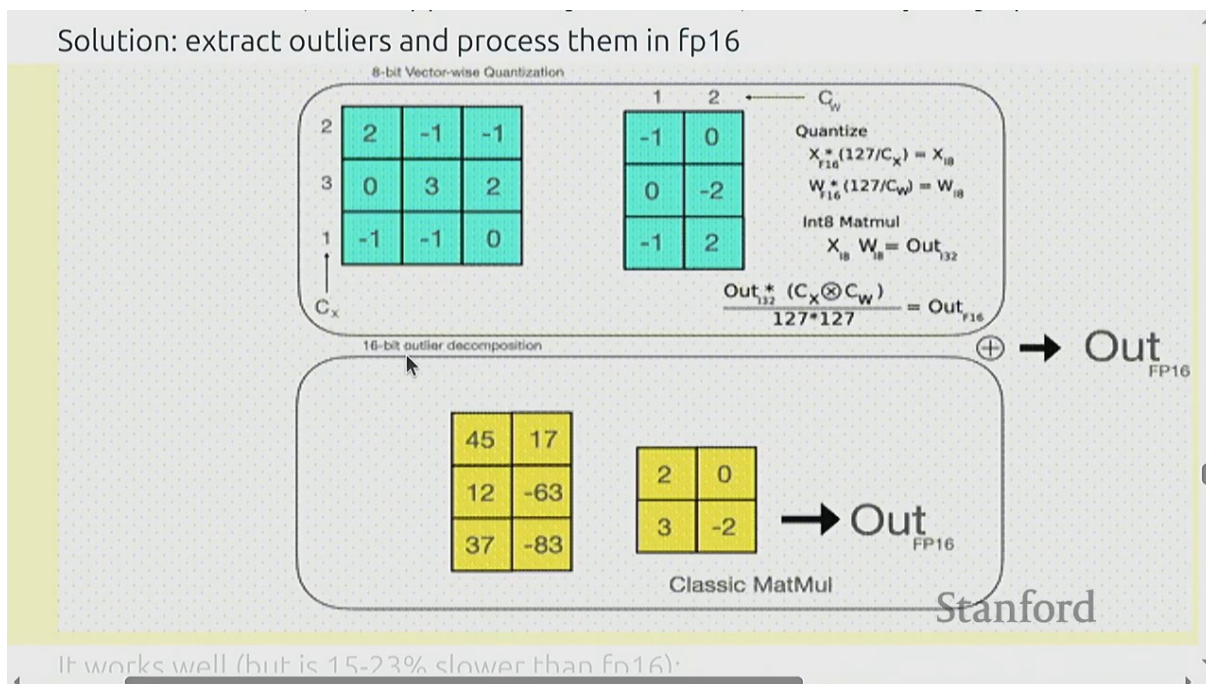


Figure 26: INT8 量化：把 BF16 权重压成 8 比特²⁶

BF16 是推理的默认精度（每个数 16 比特）。量化就是把它压成 INT8（8 比特）、甚至 INT4（4 比特）：

$$x_{int8} = \text{round}\left(\frac{x}{s}\right), \quad s = \frac{\max|x|}{127}$$

- x ：原始 BF16 权重值
- s ：缩放因子，由权重的最大绝对值决定
- x_{int8} ：量化后的 8 比特整数（-128 到 127）

量化的两倍收益

权重从 16 比特降到 8 比特：（1）显存占用减半；（2）带宽受限的解码阶段，每秒能读的权重数翻倍，吞吐直接翻倍。降到 4 比特就是 4 倍。这是性价比极高的优化。

7.1 INT8 的麻烦：异常值

但 INT8 不是白吃的午餐。问题在于：权重的分布不均匀，有些通道的值远大于其他——这些异常值（outliers）会迫使缩放因子 s 变大，结果让其他正常值被量化到只剩几个离散档位，精度严重损失。

LLM.int8() 的发现

Tim Dettmers 等人发现：LLM 里极少数通道（可能 < 0.1%）的激活值异常大，却携带了关键信息。把它们和大多数正常值一起做 INT8，等于把整批数据“按最坏情况”压缩。LLM.int8() 的解法是：把异常值挑出来单独用 BF16 算，其余做 INT8——保住了关键信息又拿到大部分加速。

7.2 AWQ：激活感知的权重量化

Activation-aware Weight Quantization (AWQ) 走另一条路：不是看权重大小，而是看哪些权重对激活更敏感。对应大激活的通道，保留更高精度；其他激进压到 4 比特。

²⁶ 视频画面时间区间：01:07:45 – 01:08:15。

7.3 蒸馏与剪枝：从大模型蒸馏到小模型

除了降精度，还可以直接换一个更小的模型：

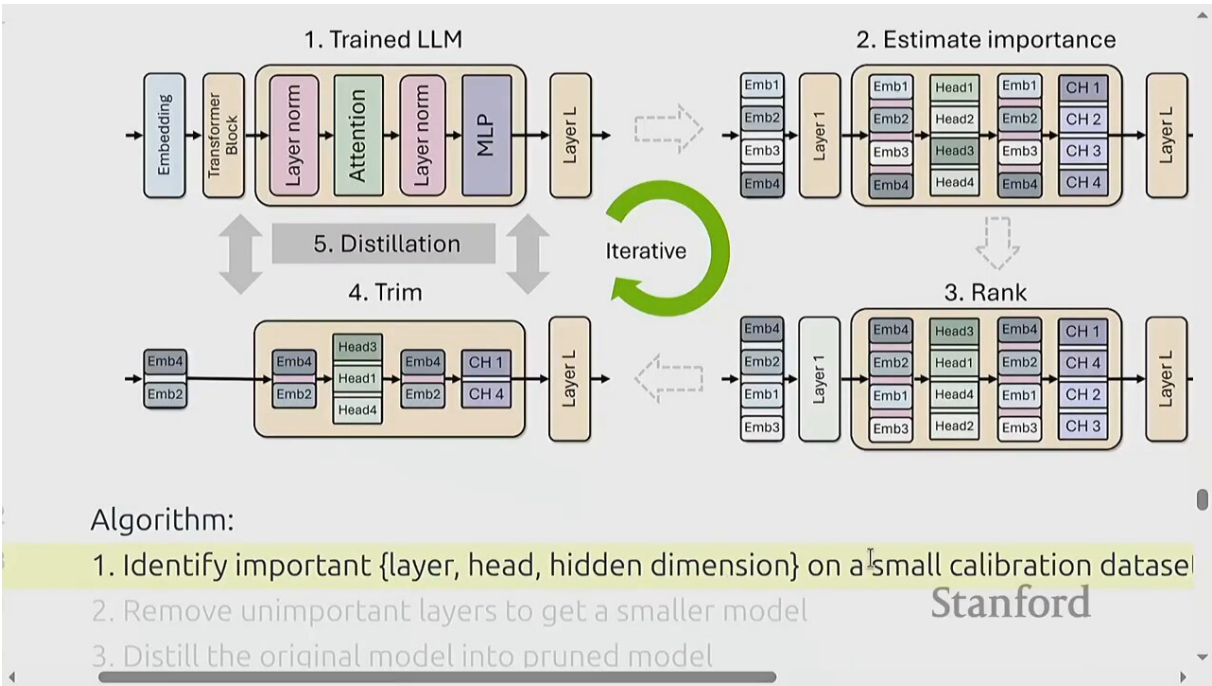


Figure 27: 蒸馏：用大模型（教师）的输出训练小模型（学生）²⁷

方法	核心理想
剪枝 (pruning)	把不重要的权重置零（结构化剪枝删整个通道），减少计算
蒸馏 (distillation)	用大模型（教师）的 logits 或中间特征当监督，训练一个更小的学生模型

为什么这些都是“lossy”

量化、剪枝、蒸馏都是有损（lossy）的——它们用一点点质量换取大幅的速度或显存收益。关键问题永远是：“这点质量损失能不能用更多训练或更聪明的算法补回来？”GPT-4、Llama 系列都是 4 比特推理，说明行业已经接受了这个权衡。

8 投机解码：用小模型猜，大模型验

到目前为止的所有优化都没改一个事实：每个 token 都要让大模型算一遍。投机解码（speculative decoding）打破这个限制——让一个小的草稿模型（draft model）先猜好几个 token，大模型再一次性验证：

²⁷ 视频画面时间区间：01:09:15 – 01:09:45。

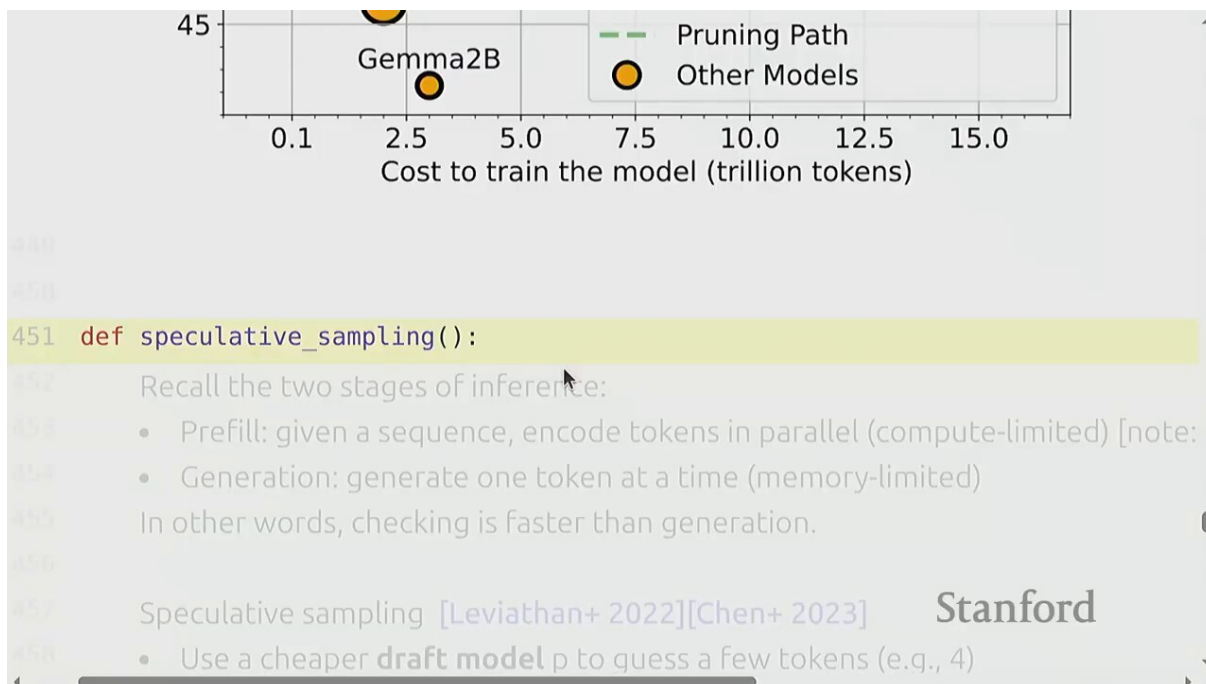


Figure 28: 投机解码：草稿模型快速生成 γ 个候选 token，目标模型一次验证²⁸

8.1 流程：草稿 + 验证

1. 草稿模型（小、快）自回归生成 γ 个候选 token: $x_1, x_2, \dots, x_\gamma$
2. 目标模型（大、准）一次性把这 γ 个 token 当输入做一次前向，得到每个位置的概率 q_t （目标分布）
3. 对比草稿模型的概率 p_t 和目标的 q_t ，决定接受哪些 token

为什么这是算力的胜利

解码阶段大模型是带宽受限、算力闲置的。投机解码让大模型一次前向同时验证多个 token——多读几个 token 几乎不增加带宽压力（反正权重都得读），却把闲置的算力用上了。只要草稿模型猜得准，吞吐就能翻倍。

8.2 拒绝采样：保证分布完全一致

最妙的是，投机解码可以用拒绝采样（rejection sampling）严格保证：最终输出的分布和直接用大模型采样完全一样。没有任何质量损失：

²⁸视频画面时间区间：01:11:15 – 01:11:45。

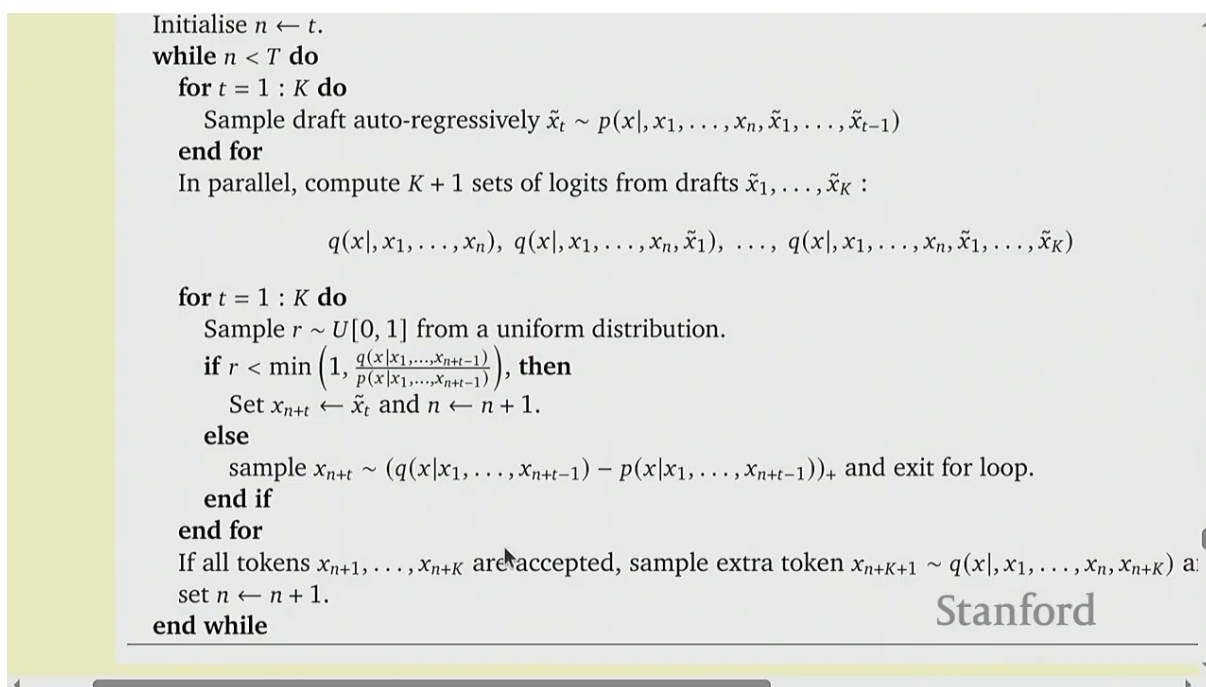


Figure 29: 投机解码 = 用 P （草稿）做提案的拒绝采样，保证最终分布与 Q （目标）完全一致²⁹

对草稿模型给出的 token x （草稿概率 $p(x)$ ，目标概率 $q(x)$ ）：

$$\text{接受概率} = \min\left(1, \frac{q(x)}{p(x)}\right)$$

- $p(x)$ ：草稿模型给出 token x 的概率
- $q(x)$ ：目标模型给 token x 的概率
- 若 $q(x) \geq p(x)$ ：草稿低估了这个 token，直接接受（接受概率 1）
- 若 $q(x) < p(x)$ ：草稿高估了，按 $q(x)/p(x)$ 的概率接受；若拒绝，按 $(q - p)^+$ 重采样

拒绝采样的数学保证

可以严格证明：经过上述接受/拒绝后，最终输出 token 的分布就是 q 。投机解码不是近似——它产出的序列和“大模型直接采样”在统计上完全等价。这是一个“既快又准”的稀有案例，理论保证了质量无损。

8.3 实证：约 2 倍加速

实测中，配一个 70 亿参数的草稿模型给 700 亿的目标模型，通常能拿到约 2 倍的吞吐提升——因为草稿模型猜得相当准，大部分 token 都被接受，大模型每步确实能“白拿”好几个 token。

草稿模型的选择

草稿模型不必和目标模型同族——只要分布够接近就行。研究热点包括：用更小的同架构模型、用知识蒸馏专门训一个“模仿者”、甚至用模型前面几层当草稿（layer-skipping）。关键权衡：草稿模型越准，接受率越高，但生成越慢；越快，每秒提议越多，但接受率越低。

9 服务层优化：批处理与内存管理

最后 Percy 转向 serving 系统层面——即便模型和架构都不变，调度策略也能大幅提升吞吐。

²⁹ 视频画面时间区间：01:14:15 – 01:14:45。

9.1 连续批处理

朴素的批处理（static batching）要等凑齐一批再一起发，发完再凑下一批。问题是请求长度不一——长请求拖累整批，短请求空等：

Summary:

- Exact sampling from target model (thanks to math)!
- Exploits asymmetry between checking and generation
- Lots of room for innovation on the draft model (involves training)

```
def continuous_batching():
```

Orca: A Distributed Serving System for Transformer-Based Generative Models[talk]

Problem:

- Training: get a dense block of tokens (batch size x sequence length)
- Inference: requests arrive and finish at different times, so you have a ragged array

The diagram shows two horizontal sequences of tokens. The first sequence is labeled $T_1, T_2, T_3, T_4, T_5, T_6, T_7, T_8$ and the second is $T_1, T_2, T_3, T_4, T_5, T_6, T_7, T_8$. Below each sequence is a row of colored boxes representing tokens. The first sequence has boxes for S_1, S_1, S_1, S_1 and then empty boxes. The second sequence has boxes for $S_1, S_1, S_1, S_1, S_1, S_1$ and then a red box labeled 'END'. The word 'Stanford' is written above the second sequence.

Figure 30: 连续批处理：请求级动态进出，不等整批³⁰

连续批处理的核心

连续批处理（continuous batching，又称 iteration-level batching）在每个解码步都重新组批：新请求随时加入、完成的请求随时退出。不再等整批——GPU 始终在满批量运转。这是 vLLM、TGI 等现代推理框架的标配。

9.2 PagedAttention：KV cache 的虚拟内存

连续批处理有个难题：每个请求的 KV cache 长度不同、还在动态变化，传统的“连续分配一大块”会导致严重的显存碎片——很多空隙用不上。vLLM 的 PagedAttention 借鉴操作系统的虚拟内存思想解决它：

³⁰视频画面时间区间：01:17:45 – 01:18:15。

/ works when all sequences have the same dimensionality (right?)
uest might have a different length

ve batching

n all sequences of the same length, operate on a $B \times S \times H$ tensor

: have different lengths: $[3, H]$, $[9, H]$, $[5, H]$, etc.

putation: process each sequence separately

n computation: concatenate all the sequences together to $[3 + 9 + 5, H]$

() :

duced vLLM in addition to PagedAttention [Kwon+ 2023]

Stanford

Figure 31: PagedAttention: KV cache 拆成固定大小的”页”，按需分配³¹

把每个请求的 KV cache 拆成固定大小的块 (block)，通过一个页表 (block table) 映射到物理显存。请求增长时按需申请新块，结束时整块释放。

类比	操作系统	PagedAttention
虚拟地址空间	进程的虚拟内存	一个请求逻辑上的连续 KV cache
物理内存	物理内存页	GPU 上固定大小的 KV block
映射	页表	block table
碎片问题	外部碎片	请求间显存碎片

PagedAttention 的三大收益

(1) 消除外部碎片：块可以放在显存任何空位，利用率接近 100%；(2) 支持变长请求：请求增长时按需申请新块，不用预留；(3) 支持共享：多个请求共享同一个前缀（如系统 prompt）的 KV 块。

9.3 写时复制：共享前缀

当很多请求共用同一段前缀（例如同一个 system prompt），没必要每个请求都存一份这段 KV。PagedAttention 用写时复制 (copy-on-write) 让它们指向同一组物理块：

³¹ 视频画面时间区间：01:19:15 – 01:19:45。

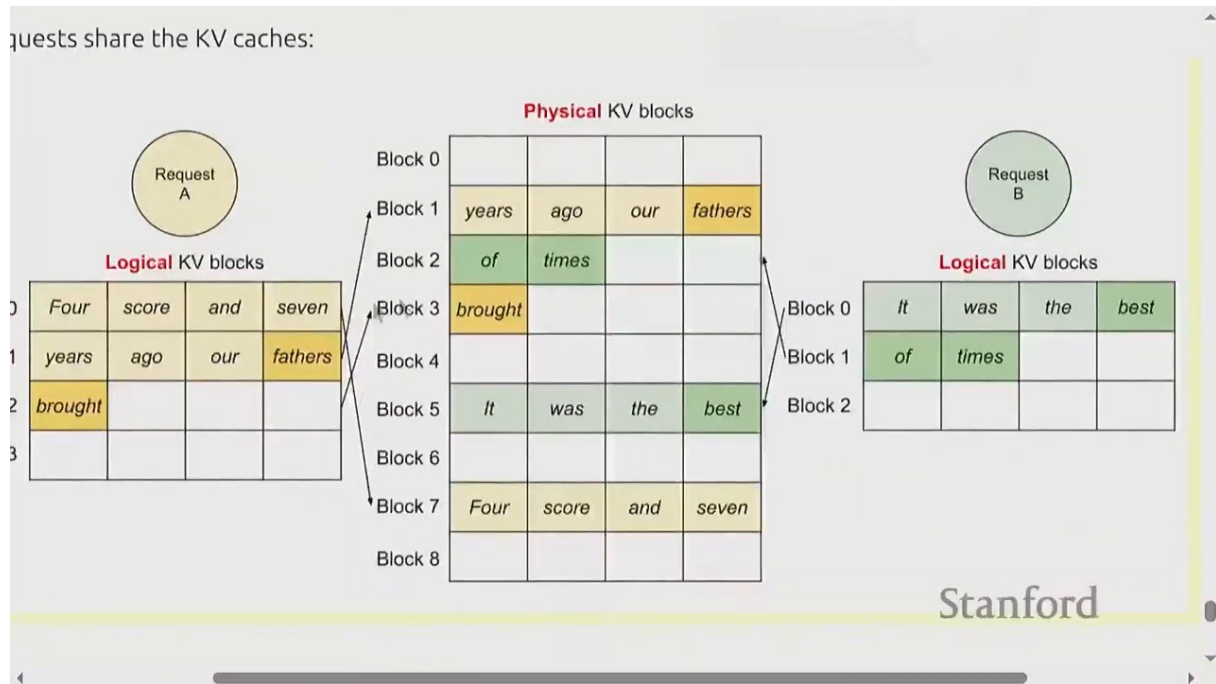


Figure 32: 写时复制：共享前缀的 KV 块，只在分歧时复制³²

- 维护每个块的引用计数（reference count），记录有多少请求在用它
- 引用计数 > 1 的块是只读共享的
- 只有当某个请求要“分叉”出自己的 KV 时，才复制那一个块（写时复制）

共享前缀的实战价值

在 agent、RAG、多轮对话场景，大量请求共享同一段长 system prompt 或长文档。共享前缀能把这段 KV 的显存和计算成本摊薄到所有请求——这是现代 LLM serving 的一个巨大效率来源。

³² 视频画面时间区间：00:20:15 – 00:20:45。

10 总结与延伸

10.1 算术强度：贯穿全讲的统一线索

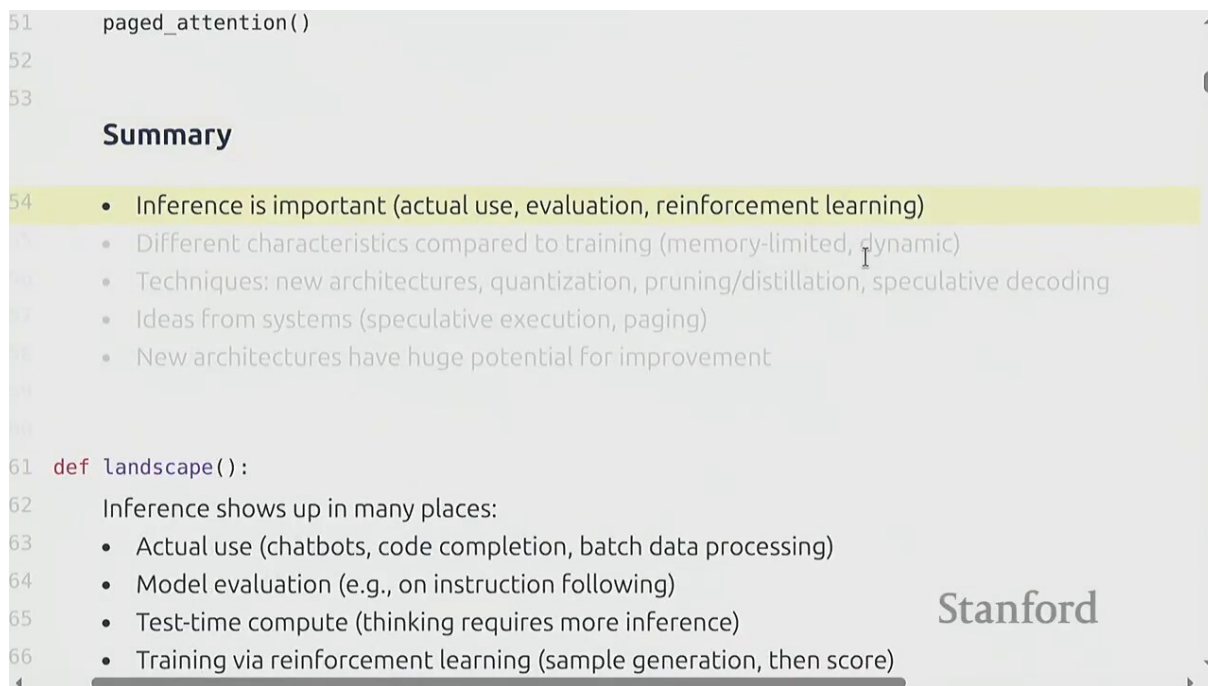


Figure 33: 总结：算术强度框架把所有推理优化技术串成一条线³³

六个核心论点

1. 算术强度是推理优化的罗盘：每个算子的 I 与 GPU 阈值 I^* 比较，立刻知道是算力受限还是带宽受限。所有优化都在围绕“提高 I ”或“省带宽”。
2. 推理分两阶段、瓶颈不同：Prefill 是 compute-bound（吃算力），Decode 是 memory-bound（吃带宽）。两阶段的优化策略完全不同。
3. $B = 1$ 是带宽地狱：单请求解码算术强度 ≈ 1 ，GPU 算力利用率不到 1/300。批量是第一救命稻草。
4. KV cache 是显存天花板：随序列线性增长，是批量大小的硬限制。GQA/MLA/SSM 都在攻击它。
5. 量化是性价比之王：4 比特推理在工业界已是标配，吞吐直接翻几倍，质量损失可接受。
6. 投机解码是稀有的“无损加速”：用拒绝采样保证分布不变，实测 2 倍提速。理论保证 + 工程收益，这是最佳实践。

³³ 视频画面时间区间：01:21:15 – 01:21:45。

10.2 技术地图回顾

优化方向	代表技术	攻击的瓶颈
改架构	GQA, MLA, CLA, 局部注意力	KV cache 体积
换范式	SSM/Mamba, 线性注意力, 扩散 LLM	$O(N^2)$ 或 $O(N)$ 增长
降精度	INT8/ INT4, LLM.int8(), AWQ	带宽、显存
压缩模型	剪枝、蒸馏	算力、显存
算法层	投机解码	闲置算力（带宽受限时）
服务层	连续批处理, PagedAttention, 共享前缀	碎片、空转、冗余存储

10.3 下节预告

后续讲座会进一步讨论训练系统的并行策略（数据并行、张量并行、流水线并行、FSDP）以及长上下文与MoE（混合专家）架构——它们既是训练的关键技术，也深刻影响推理时的成本结构。

10.4 配套阅读

以下论文/系统建议按本讲主题阅读：

论文/系统	对应知识点
GQA (Ainslie 2023)	Grouped-Query Attention
DeepSeek-V2/V3 技术报告	MLA 多头潜在注意力
Mamba (Gu & Dao 2023)	状态空间模型
vLLM (Kwon 2023)	PagedAttention, 连续批处理
Speculative Decoding (Leviathan 2023)	投机解码 + 拒绝采样
LLM.int8() (Dettmers 2022)	异常值感知的 INT8 量化
AWQ (Lin 2023)	激活感知权重量化
FlashAttention (Dao 2022)	IO 感知的精确注意力